

Análisis de Procedimiento Almacenado

Prueba Técnica Fullstack — Tecnoglass / Energía Solar S.A. | Procedimiento evaluado: **sp_ResumenCarteraClientes**
| Punto 3 — Análisis de eficiencia y concurrencia

El SP calcula, por cliente, el total de facturas, las facturas vencidas y el porcentaje de mora, usando un cursor externo (clientes) que anida un cursor interno (facturas) y, para cada factura, una subconsulta TOP 1 sobre el historial. Este patrón fila-por-fila (RBAR) presenta fallas críticas de escalabilidad y de manejo transaccional.

Problemas Críticos Identificados

- **Cursores anidados (RBAR):** Cursor externo sobre Cliente + cursor interno sobre Factura generan complejidad efectiva $O(N \times M)$ (clientes $\times$ facturas), con overhead de apertura/cierre de cursor en cada iteración. No escala con el crecimiento de la cartera.
- **Subconsulta repetida por fila:** El `SELECT TOP 1 ... ORDER BY FechaMovimiento DESC` sobre `HistorialFactura` se ejecuta una vez por cada factura individual, multiplicando lecturas a disco/buffer en lugar de resolverse en una sola pasada sobre el conjunto.
- **Ausencia de control transaccional:** No se define `BEGIN TRAN` ni nivel de aislamiento. En un entorno con escrituras concurrentes a `Factura`/`HistorialFactura`, el cálculo puede leer datos inconsistentes mientras recorre lentamente cada cliente (riesgo de "phantom reads").
- **Inserciones fila por fila en tabla temporal:** Cada resultado se inserta individualmente en `#Resumen` en vez de poblarse mediante una sola operación de conjunto (`INSERT...SELECT`), sumando latencia de E/S innecesaria.
- **Dependencia implícita de índices no garantizados:** El patrón de acceso (último estado por factura, conteo por cliente) requiere índices sobre `Factura(ClientId)` y `HistorialFactura(FacturaId, FechaMovimiento DESC)` que el script no garantiza.

Propuesta de Optimización (Set-Based SQL)

- **Eliminar ambos cursores con CTE + ROW_NUMBER():** Particionar por `FacturaId` y ordenar por `FechaMovimiento DESC` para obtener el último estado de cada factura en una sola pasada, sin iteración fila por fila.
- **Resolver con un único JOIN + GROUP BY:** Unir `Cliente`, `Factura` y la CTE de "último estado"; calcular `TotalFacturas` y `FacturasVencidas` con `COUNT(*)` y `SUM(CASE WHEN Estado='VENCIDA' THEN 1 ELSE 0 END)`, agrupando por `ClientId` en una sola operación basada en conjuntos.
- **Nivel de aislamiento adecuado para lectura consolidada:** Usar `READ COMMITTED` (o `SNAPSHOT` si se requiere evitar bloqueos) ya que es una operación de solo lectura; no se necesita un nivel más estricto ni bloqueos exclusivos.
- **Reemplazar inserciones individuales por un único SELECT/INSERT...SELECT:** Aprovechar que las agregaciones ya se resuelven en conjunto para poblar el resultado en una sola operación.
- **Soportar con índices (no creados aquí, solo recomendados):** Índices sobre `Factura(ClientId)` y `HistorialFactura(FacturaId, FechaMovimiento DESC)` para que el optimizador resuelva el JOIN vía Index Seek en lugar de Table Scan.

Impacto esperado: al pasar de un patrón fila-por-fila $O(N^2)$ a una solución basada en conjuntos respaldada por índices, el optimizador de SQL Server puede resolver la consulta mediante uniones hash/merge en una sola pasada, reduciendo drásticamente el tiempo de ejecución y el riesgo de inconsistencias en cargas concurrentes.

Documento de análisis — Punto 3 de la prueba técnica. No se reescribió ni ejecutó el script original; no se crearon índices físicos, conforme a las instrucciones de la convocatoria.